Implementing the Simple C Compiler

Gustavo Rodriguez-Rivera
Purdue University
West Lafayette, Indiana, USA
grr@cs.purdue.edu

Max Jacobson
Purdue University
West Lafayette, Indiana, USA
jacobs57@purdue.edu

ACM Reference Format:
Gustavo Rodriguez-Rivera and Max Jacobson . 2023. Implementing the Simple C Compiler. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 THE SIMPLE-C LANGUAGE

Simple-C is a subset of the C language with some restrictions. First, it supports only variable types long, char*, long* char **, and void, all using 8 bytes, and arrays, simplifying code generation. The language supports all the constructions "if/else", "while", "for", "do-while", "break", "continue", etc. Being a subset of "C", it allows interoperation with the C-run-time library as long as the calls use long and char*. Simple-C programs are written as a list of global variables and functions, similar to C programs, and also use the ".c" extension. Programs compiled with the Simple-C compiler can be tested by comparing the output of programs of the same code compiled with a regular C compiler. The following is an example program written in Simple-C.

```
long sum(long n, long* a) {
    long i, s;
    s = 0;
    for (i=0; i<n; i=i+1) {
        s = s + a[i];
    }
    return s;
}

long main()
{
    long* a;
    a = malloc(5*8);
    a[0]=4; a[1]=3; a[2]=1; a[3]=7; a[4]= 6;
    long s;
    s = sum(5, a);
    printf("sum=%d\n", s);
}
```

There is no type-checking since the project concentrates only on code generation, but it could be extended. When a char* is used as a long value, the equivalent memory address will be used instead. That is possible to do since all variables are 8 bytes long – essentially, automatic type casting.

2 GRAMMAR AND PARSING

Modern compilers separate the compilation processing into two stages called Scanner and Parser. The Simple-C compiler uses Lex and Yacc, the UNIX standard tools, to generate the Scanner and Parser. The Scanner first puts together the characters in a program

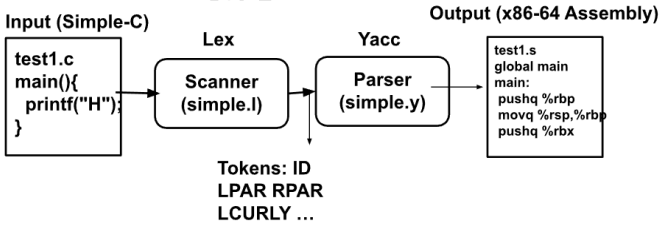

Figure 1: Scanner and Parser. The Scanner separates the program into tokens. In our compiler, the Parser generates the assembly file directly.

into "tokens" that represent language keywords such as "if", left parenthesis "(", right parenthesis ")", integer constants, strings, identifiers, etc. Lex receives as an input file a list of regular expressions that describe the tokens, and it generates the Scanner. When the Scanner reads an input program, it returns the token type of the regular expression that best matches the token. Here is an example of some tokens described in this file.

```
"("     {
                return LPARENT;
        }
"if"    {
                return IF;
        }
[A-Za-z][A-Za-z0-9]*  {
                yylval.string_val = strdup(yytext);
                return WORD;
        }
-?[0-9][0-9]*  {
                yylval.string_val = strdup(yytext);
                return INTEGER_CONST;
        }
```

The second stage, called Parser, uses grammar rules to describe the program's syntax. The Simple-C compiler grammar is implemented in a Yacc file, "simple. y", that describes a subset of the C language as a list of global variables and function declarations. In our project, the grammar is given to the students, so they can only concentrate on the code generation part, adding actions to write the assembly

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

instructions in a file used to generate the executable. The actions are C/C++ code between curly braces "{...}" embedded inside the grammar. These actions are called when the Parser identifies these constructions.

Here is part of the Simple-C grammar that describes a function definition.

```
function:
     var_type WORD
     {
        fprintf(fasm, "\t.text\n");
        fprintf(fasm, ".globl %s\n", $2);
        fprintf(fasm, "%s:\n", $2);
        fprintf(fasm, "\t#Saving frame pointer\n");
        fprintf(fasm, "\tpushq %%rbp\n");
        fprintf(fasm, "\tmovq %%rsp,%%rbp\n");
     }
     LPARENT arguments RPARENT compound_statement
     {
        fprintf(fasm, "\tleave\n");
        fprintf(fasm, "\tret\n");
     }
    ;
```

When a main function definition such as the following is given in Simple-C, the grammar will match the function grammar rule, and it will run the C code that generates the prologue of the function to make the "main" identifier global and add "main:" to mark the beginning of the function.

```
test1.c:
int main() {
 ...
}
```

The "simple.y" Parser also will generate the assembly code to save the frame pointer when the function starts.

```
test1.s:
        .text
.globl main
main:
        #Saving frame pointer
        pushq %rbp
        movq %rsp,%rbp
        ...
        leave
        ret
```

During the compiler project, the students need first to understand the basics of scanning and parsing and be able to build a parse tree given grammar and input. Then they need to understand the Simple-C grammar and add actions to the grammar to generate the different parts of the program. Before the compiler project, the students have two assignments to familiarize themselves with assembly language programming.

3 GENERATING CODE FOR EXPRESSIONS

To understand code generation for expressions, we first need to understand Reverse-Polish-Notation or RPN. RPN execution or stack-based evaluation uses a stack of numbers to perform the evaluation that we call Expression Stack.

An expression such as $4+3*8$ is equivalent to the expression in RPN: $4, 3, 8, *, +$, where when a number is found, it is pushed in the Expression Stack, when an operation such as "+" or "*" will pop two items from the Expression Stack add or multiply them and push the result in the Expression Stack.

```
4    push 4              Stack=(4}
3    push 3              Stack=(4,3}
8    push 3              Stack=(4,3,8}
"*"  push(pop()+pop())   Stack=(4,24}
"+"  push(pop()*pop())   Stack={28}
```

Generating code for an Expression Stack is relatively easy for a Parser since actions can be placed after identifying a number token, an addition, and a multiplicative expression.

```
additive_expr:
         multiplicative_expr
         | additive_expr PLUS multiplicative_expr {
          fprintf(fasm,"\n\t# PUSH(POP()+POP())\n");
         };
multiplicative_expr:
         primary_expr
         | multiplicative_expr TIMES primary_expr {
          fprintf(fasm,"\n\t# PUSH(POP()*POP())\n");
         };
primary_expr:
        INTEGER_CONST {
                fprintf(fasm, "\n\t# PUSH(%s)\n",
                $<string_val>1);
        }
```

An expression such as "4+3*8" as input to the Parser will generate the output:

```
#  PUSH(4)
#  PUSH(3)
#  PUSH(8)
#  PUSH(POP()*POP())
#  PUSH(POP()+POP())
```

The input will generate the parse tree in Figure 2. and the actions will be executed as an in-order traversal of the parse tree. Some

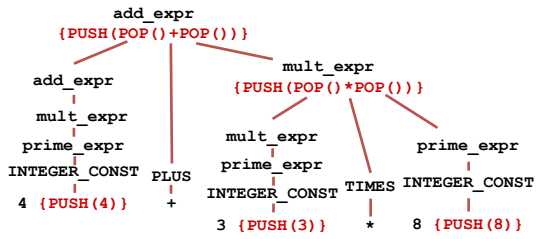

Figure 2: Parse Tree for the input "4 + 3 * 8"

compilers use the Execution Stack of the program, which stores return addresses, local variables, etc., to implement the PUSH/POP operation in the Expression Stack. However, this has the disadvantage that it requires access to RAM, which is about two orders of

magnitude slower than the registers. Our compiler optimizes the Expression Stack access by placing the bottom elements in registers. For instance, in the x86-64 architecture, the registers rbx, r10, r13, r14, and r15, are available to the function as long as they are saved and restored at the beginning of the function that uses them. The compiler uses these registers to implement the entries at the base of the stack. When the compiler runs out of these registers, it will push the registers to the Execution Stack. Since most expressions are shallow, most of the time the Execution stack will not be needed during expression evaluation.

Stack Position	Register/Memory
0	rbx
1	r10
2	r13
3	r14
4	r15
>=5	Use Execution Stack

For example, assuming the Register Stack is empty, the top equals 0. The PUSH(4) operation will be generated as "movq $4, %rbx" and top equals 1. The next instruction PUSH(3), will be translated as "movq $3, %r10" and top equals 2. When the next instruction is PUSH(8), it will be translated as "movq $8, %r13" and top equals 3. The instruction PUSH(POP()*POP()) will multiply the two items at the top of the stack and place it at top-2, and it will be translated to "imulq %r13, %r10", top equals 2, and the result is placed at %r10. The instruction PUSH(POP()+POP()) will add the two items at the top of the stack and place it at top-2, and it will be translated to "addq %r10, %rbx", and top equals 1. The result will be left at the base of the stack at rbx.

Stack Ops	Assembly Ops	Stack
PUSH(4)	movq $4, %rbx	{4}
PUSH(3)	movq $3, %r10	{4,3}
PUSH(8)	movq $8, %r13	{4,3,8}
PUSH(POP()*POP())	imulq %r13, %r10	{4,24}
PUSH(POP()+POP())	addq %r10, %rbx	{28}

The result of an expression will always be at the bottom of the stack in register rbx, where it can be used by"if", "while", or other statements that use expressions. The portion of the Parser actions that generate machine code for expressions is described below.

```
additive_expr:
  multiplicative_expr
  | additive_expr PLUS multiplicative_expr {
        fprintf(fasm,"\n\t#  PUSH(POP()+POP())\n");
        if (top<nregStk) {
                fprintf(fasm, "\taddq %%%s,%%%s\n",
                        regStk[top-1], regStk[top-2]);
                top--;
        }
        else {error("Save regs not implemented"); }
  };
multiplicative_expr:
  primary_expr
  | multiplicative_expr TIMES primary_expr {
        fprintf(fasm,"\n\t#  PUSH(POP()*POP())\n");
        if (top<nregStk) {
                fprintf(fasm, "\timulq %%%s,%%%s\n",
                        regStk[top-1], regStk[top-2]);
                top--;
        }
        else {error("Save regss not implemented"); }
  };
primary_expr:
 INTEGER_CONST {
      fprintf(fasm, "\n\t# PUSH(%s)\n", $<string_val>1);
        if (top<nregStk) {
                fprintf(fasm, "\tmovq $%s,%%%s\n",
                        $<string_val>1, regStk[top]);
                top++;
        }
        else {error("Save regss not implemented"); }
  }
```

4 IMPLEMENTING VARIABLES

The compiler implements three types of variables: global, local, and arguments. Global variables are stored in the program's DATA segment and can be referenced by a global name. Local variables are stored in the Execution Stack in the activation record of the current function and can be referenced using the frame pointer register rbp. Arguments can be handled as local variables that take the value of the arguments as the initial value.

The following portion of the Parser allocates space for global variables:

```
Simple C:
   long g;
Assembly:
   .data
   .comm g, 8
Parser segment that generates global variables:
   global_var:
       var_type global_var_list SEMICOLON;
   global_var_list: WORD {
       char * var_name = $1;
       fprintf(fasm,"\t.data\n");
       fprintf(fasm,"\t.comm %s, 8\n", var_name);
   }
   | global_var_list COMA WORD {
       ...
   };
   var_type: CHARSTAR | CHARSTARSTAR | LONG
     | LONGSTAR | VOID;
```

The following example fetches a global variable, g, increments it, and then stores them in another global variable, r.

```
Simple C:
   g = r+8;
Assembly:
    movq $8,%rbx
    movq r,r10
    addq %r10,%rbx
    movq %rbx,g
Parser segment that generates global variables:
   primary_expr:
```

```
        ...
        | WORD {
            // Assume it is a global variable
            char * id = $<string_val>1;
            fprintf(fasm, "\tmovq %s,%%%s\n", id,
              regStk[top]);
            top++;
        }
        ...
    ;
    assignment:
        WORD EQUAL expression {
            char * id = $1;
            fprintf(fasm, "\tmovq %%rbx,%s\n", id);
            top = 0;
        }
        ...
    ;
```

Local variables are stored in the stack, and the function's arguments will also be stored in the stack. We need to reserve stack space at the beginning of the function using

```
subq $<space>, %rsp
```

Where <space> is the space reserved for the local variables and arguments in the stack, and that needs to be restored using "addq $<space>, %rsp" before leaving the function. The problem is that since local variables can be stored anywhere in the function, we will know how much space to reserve for local variables at the end of the function. By design, this compiler is a one-pass compiler because the Parser generates the code directly while parsing. To solve this problem, after parsing the function, it writes a constant in an included file, the number of local variables and arguments that the function uses. The file will be included at the top of the generated assembly output, and it will be included when the file is assembled to generate the executable.

When compiling a program, an array of global variable names stores the global variable definitions. Also, when compiling a function, an array of local variable names is created, and the arguments are added to the table. Arguments are copied to the corresponding location in the stack when the function starts. Also, the variable name is added to the table whenever there is a local variable definition. If an expression contains the name of a variable, it is first looked up in the local variables array. If it is found, the index is used to compute the offset of the variable relative to the frame pointer in rbp. If it is a global variable, then the variable's name is used as the address of the variable.

5 CONDITIONAL STATEMENTS AND LOOPS

To implement conditional statements such as `while ([expression]) [statement]`, the parser will generate code as follows:

```
while_1:
    ... Code for expression...
   cmpq $0, %rbx  # Compare the top of the stack with 0
    je after_while_1 # Jump after while loop if false
    ... Code for statement ...
   jmp while_1   # Jump back to the beginning of while loop
after_while_1:     # End of while
```

Execution Stack

%rbp	...
-8	a
-16	b
-24	c
-32	d

```
long add(long a, long b){
   int c,d;
   c=5;
   d=c+a*b;
   return d;
}
```

Figure 3: Implementing Local Variables and Arguments using Execution Stack and Frame Pointer register (rbp).

The Parser generates a label *while*_1 at the top of the loop just before evaluating the expression. The expression result will be left at the top of the stack, register rbx, which is compared with 0 and jumps to label *after_while*_1 if it is false. The label number will be increased for every while loop and attached to the non-terminal part of the grammar rule.

6 CONCLUSIONS

Linus Torvalds, the creator of Linux, once said, "When I read C, I know what the assembly language will look like, and that is something I care about." this project gives the student this knowledge. We show the Simple-C compiler project and a register-stack algorithm to convert expressions into assembly code. It also explained how it can be included in a Computer Architecture class and how the students can complete it successfully. By completing the project, the students experience how high-level languages are translated to machine code and how t runs in the underlying hardware.

REFERENCES